

Botón con Raspberry Pi Pico

Resumen completo de la conversación · 5 de octubre de 2026

1. El punto de partida

Proyecto en Wokwi con una Raspberry Pi Pico, escrito en C++ con el SDK de la Pico. Tiene una clase **Boton** repartida en tres archivos. El objetivo: agregar un segundo botón con otra función, que encienda el LED integrado de la placa.

main.cpp (original)

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "Boton.h"

int main() {
    stdio_init_all();
    Boton miPrimerBoton(7);

    while (true) {
        miPrimerBoton.leer();
        if (miPrimerBoton.presionado) {
            printf("bacan estoy presionado, pero igual me presiona\n");
        }
        else {
            printf("no hay nadie presionandome\n");
        }
        sleep_ms(10);
    }
}
```

Boton.h

```
#ifndef BOTON_H
#define BOTON_H
#include "hardware/gpio.h"

class Boton {
public:
    bool presionado = false;
    int duracionPresionado = 0;
    int patita;

    Boton(int nuevaPatita);

    void leer();
    void presionar();
    void soltar();
};

#endif
```

Boton.cpp (original)

```
#include "Boton.h"

Boton::Boton(int nuevaPatita) {
    Boton::patita = nuevaPatita;
    gpio_init(Boton::patita);
    gpio_set_dir(Boton::patita, GPIO_IN);
}
```

```

}

void Boton::leer() {
    Boton::presionado = gpio_get(Boton::patita);
}

void Boton::presionar() {
    Boton::presionado = true;
    // queda pendiente calcular cuanto rato lleva presionado
}

void Boton::soltar() {
    Boton::presionado = false;
    Boton::duracionPresionado = 0;
}

```

2. Agregar el segundo botón y el LED

No hace falta tocar **Boton.h** ni **Boton.cpp** para esto: la clase ya se puede reutilizar. Solo se crea un segundo objeto en **main.cpp** y se usa su estado para controlar el LED. En Wokwi hay que agregar un segundo pulsador y conectarlo a otro pin GP.

main.cpp con comentarios detallados

```

// libreria estandar de C para usar printf
#include <stdio.h>
// libreria base del SDK de la Pico (trae sleep_ms, gpio, etc.)
#include "pico/stdlib.h"
// nuestra clase Boton
#include "Boton.h"

int main() {

    // activa la comunicacion por USB/serial
    // sin esto, los printf no se ven en el monitor serial
    stdio_init_all();

    // ----- LED INTEGRADO -----

    // PICO_DEFAULT_LED_PIN es una constante del SDK (vale 25):
    // el pin donde viene soldado el LED de la placa
    // "const": este valor nunca cambia
    // "uint": entero sin signo (solo numeros positivos)
    const uint LED = PICO_DEFAULT_LED_PIN;

    // prepara el pin para usarlo como GPIO normal
    gpio_init(LED);

    // el LED es una SALIDA: la Pico le manda voltaje
    // (el boton, en cambio, es ENTRADA: la Pico escucha el voltaje)
    gpio_set_dir(LED, GPIO_OUT);

    // ----- BOTONES -----

    // cada "Boton nombre(pin)" llama al constructor, que guarda el pin
    // y lo configura como entrada
    // son dos objetos distintos, cada uno con su pin y su "presionado"
    Boton miPrimerBoton(7); // boton que solo imprime mensajes
    Boton botonLed(15);     // boton que controla el LED

    // ----- LOOP PRINCIPAL -----
    while (true) {

```

```

// cada boton revisa su pin y actualiza "presionado"
// hay que llamar leer() en cada vuelta para mantener el estado al dia
miPrimerBoton.leer();
botonLed.leer();

// --- primer boton: solo mensajes ---
if (miPrimerBoton.presionado) {
    printf("bacan estoy presionado, pero igual me presiona\n");
}
else {
    printf("no hay nadie presionandome\n");
}

// --- segundo boton: controla el LED ---
// gpio_put(pin, valor) prende el pin si valor es true
// y lo apaga si es false. Como "presionado" ya es true/false:
//  boton presionado    -> LED prendido
//  boton sin presionar -> LED apagado
gpio_put(LED, botonLed.presionado);

// espera 10 ms antes de la siguiente vuelta
sleep_ms(10);
}
}

```

Constructor de Boton.cpp con pull-down

```

Boton::Boton(int nuevaPatita) {

    // guardar el numero de pin en el atributo "patita" de este objeto
    Boton::patita = nuevaPatita;

    // preparar el pin como GPIO normal
    gpio_init(Boton::patita);

    // el pin es ENTRADA: la Pico solo lee su voltaje
    gpio_set_dir(Boton::patita, GPIO_IN);

    // resistencia pull-down interna de la Pico
    // un pin de entrada "al aire" es flotante y puede leer 0 o 1 al azar
    // el pull-down lo conecta suavemente a GND (0V), asi que:
    //  boton sin presionar -> el pin lee 0 (false), estable
    //  boton presionado    -> 3V3 le gana al pull-down y lee 1 (true)
    // OJO: asume que el boton esta conectado entre el pin y 3V3
    gpio_pull_down(Boton::patita);
}

```

3. Preguntas que surgieron

¿Por qué no usar LED_BUILTIN?

LED_BUILTIN es una constante del framework de Arduino. Este proyecto usa el SDK de C/C++ de la Pico, donde la constante equivalente es **PICO_DEFAULT_LED_PIN** (GP25 en la Pico normal). Si se prefiere el nombre estilo Arduino, se puede definir:

```
#define LED_BUILTIN PICO_DEFAULT_LED_PIN
```

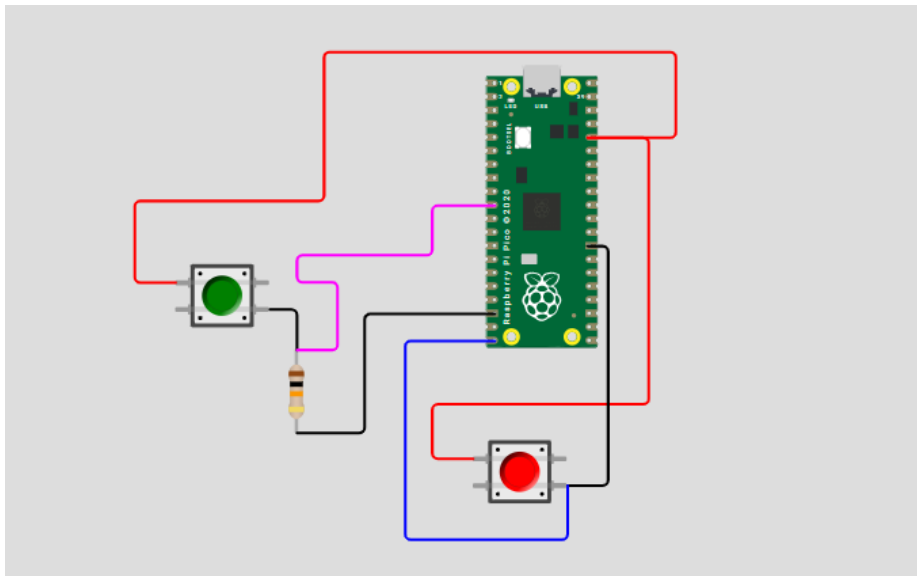
En una **Pico W** el LED está conectado al chip WiFi y se controla distinto (con `cyw43_arch_gpio_put`).

¿A qué otro pin se puede pasar el botón?

- **Se pueden usar:** GP2 a GP22, y también GP26, GP27 y GP28.
- **Evitar:** GP7 (ya lo usa el primer botón), GP25 (LED integrado), GP23 y GP24 (uso interno de la Pico), y GP0 y GP1 (UART por defecto, pueden mezclarse con printf).
- **Ojo con la numeración:** en el código se usa el número GP (GP15 = 15), que no es el número físico del pin en la placa (GP15 está en el pin físico 20).

4. El problema en el simulador

El segundo botón se comportaba igual que el primero y el LED se encendía un instante al soltarlo. Al revisar la captura del circuito de Wokwi se vio la causa:



- Del mismo lado del pulsador rojo salían **dos cables**: el azul (al pin GP) y uno negro directo a GND.
- Al presionar el botón, 3V3 y GND quedaban unidos: un **cortocircuito**, de ahí el comportamiento raro.
- El primer botón (verde) estaba bien: usa una resistencia externa como pull-down. El segundo no la necesita porque la Pico trae una interna (`gpio_pull_down`).

Solución: borrar el cable negro. El botón debe quedar solo con un cable a 3V3 y otro al pin GP. También hay que verificar que el número de pin en el código coincida con el pin donde está conectado el cable azul (GP15 en este caso).

Resultado: con el cable negro eliminado, todo funcionó correctamente.

5. LED encendido por un tiempo determinado

Nuevo objetivo: en vez de que el LED esté prendido solo mientras se mantiene pulsado el botón, que se encienda por una cantidad de tiempo fija después del toque. Para eso el **Boton** debe «acordarse» de cuándo fue presionado. Se agregan **dos atributos** y **un método**, y se ajusta `leer()`.

Boton.h (version nueva)

```

#ifndef BOTON_H
#define BOTON_H

// ahora tambien necesitamos "pico/stdlib.h" para poder medir el tiempo
// (trae get_absolute_time y to_ms_since_boot)
#include "pico/stdlib.h"
#include "hardware/gpio.h"

class Boton {

public:

    // atributos
    bool presionado = false;
    int duracionPresionado = 0;
    int patita;

    // NUEVO: en que momento (en milisegundos desde que se encendio la Pico)
    // se apreto el boton por ultima vez
    uint32_t momentoToque = 0;

    // NUEVO: dice si hay un temporizador corriendo
    // sirve para distinguir "nunca lo han apretado" de "lo apretaron hace rato"
    bool temporizadorActivo = false;

    // constructor
    Boton(int nuevaPatita);

    // metodos
    void leer();
    void presionar();
    void soltar();

    // NUEVO: devuelve true mientras no haya pasado el tiempo indicado
    // desde el ultimo toque
    bool temporizadorVigente(uint32_t duracionMs);

};

#endif

```

Boton.cpp: leer() modificado y metodo nuevo

```

void Boton::leer() {

    // guardamos como estaba el boton ANTES de leer de nuevo
    bool estabaPresionado = Boton::presionado;

    // leer boton
    Boton::presionado = gpio_get(Boton::patita);

    // detectar el momento exacto del toque:
    // ahora esta presionado, pero antes NO lo estaba
    // (asi, si mantienes apretado, el tiempo no se reinicia en cada vuelta)
    if (Boton::presionado && !estabaPresionado) {

        // get_absolute_time() es "la hora actual" segun la Pico
        // to_ms_since_boot convierte esa hora a milisegundos
        Boton::momentoToque = to_ms_since_boot(get_absolute_time());

        // arrancamos el temporizador
        Boton::temporizadorActivo = true;
    }
}

```

```
// NUEVO
bool Boton::temporizadorVigente(uint32_t duracionMs) {

    // si nunca se ha apretado (o ya se acabo el tiempo), no hay nada que hacer
    if (!Boton::temporizadorActivo) {
        return false;
    }

    // que hora es ahora, en milisegundos
    uint32_t ahora = to_ms_since_boot(get_absolute_time());

    // tiempo transcurrido desde el toque
    // si ya paso la duracion pedida, apagamos el temporizador
    if (ahora - Boton::momentoToque >= duracionMs) {
        Boton::temporizadorActivo = false;
        return false;
    }

    // todavia estamos dentro del tiempo
    return true;
}
```

main.cpp: cambios

```
// cuanto rato queda prendido el LED, en milisegundos (3000 = 3 segundos)
const uint32_t TIEMPO_LED = 3000;

// ... dentro del while ...

// antes: gpio_put(LED, botonLed.presionado);
// ahora: el LED esta prendido mientras el temporizador siga vigente
gpio_put(LED, botonLed.temporizadorVigente(TIEMPO_LED));
```

Cómo se comporta

- Al apretar el botón el LED se prende, aunque se suelte enseguida, y se apaga a los 3 segundos del toque.
- Si se aprieta de nuevo antes de que termine, el tiempo **se reinicia** desde ese nuevo toque.
- Si se mantiene apretado, el tiempo no se reinicia en cada vuelta del loop: cuenta desde el primer instante.
- miPrimerBoton no se ve afectado: ejecuta el nuevo leer (), pero nunca llama a temporizadorVigente.
- Se usan dos atributos porque hace falta saber si el botón **nunca** ha sido presionado; si no, el LED se prendería los primeros 3 segundos sin que nadie apretara nada.
- Para cambiar la duración basta con modificar TIEMPO_LED.

6. Errores de compilación y cómo se corrigieron

Al probar el código nuevo en Wokwi, la compilación falló. Eran tres errores pequeños; el resto de los mensajes eran consecuencia de ellos.

1. Faltaba el include en Boton.h

Los errores `get_absolute_time was not declared` y `to_ms_since_boot was not declared` significan que el compilador no ve la librería de tiempo del SDK. Se arregla con:

```
#include "pico/stdlib.h"
#include "hardware/gpio.h"
// si sigue fallando, agregar tambien de forma explicita:
#include "pico/time.h"
```

2. Error de tipeo en Boton.cpp (linea 58)

```
bool Boton::temporizadorVigente(uint32_t duracionMs) {    // mal
bool Boton::temporizadorVigente(uint32_t duracionMs) {    // bien
```

Con «Boton» el compilador no reconocía la clase y tomaba la función como una función suelta, lo que provocó el mensaje `invalid use of non-static data member`.

3. Faltaba un punto y coma (linea 70)

```
uint32_t ahora = to_ms_since_boot(get_absolute_time())    // mal
uint32_t ahora = to_ms_since_boot(get_absolute_time());   // bien
```

El aviso `control reaches end of non-void function` también desaparece al corregir esto, porque el compilador se perdía al leer la función.

7. Próximos pasos posibles

- Alternar el LED con cada toque (un toque prende, otro apaga).
- Implementar `duracionPresionado`, que quedó pendiente, para saber cuánto lleva presionado un botón.
- Agregar un tercer botón con otra función.